❹ Defines the expected behavior for the OrderService mock bean

❺ Expects the order is created successfully

Next, run the slice tests for the web layer to ensure they pass:

```
$ ./gradlew test --tests OrderControllerWebFluxTests
```

Great job! You successfully built and tested a reactive application, maximizing scalability, resilience, and cost-effectiveness. In the source code accompanying the book, you can find more test examples, including full integration tests using the `@SpringBootTest` annotation and slice tests for the JSON layer using `@JsonTest`, as you learned in chapter 3.

Polar Labs

Feel free to apply what you learned in the previous chapters and prepare the Order Service application for deployment.

1. Add Spring Cloud Config Client to Order Service to make it fetch configuration data from Config Service.
2. Configure the Cloud Native Buildpacks integration, containerize the application, and define the commit stage of the deployment pipeline.
3. Write the Deployment and Service manifests for deploying Order Service to a Kubernetes cluster.
4. Configure Tilt to automate the Order Service deployment to your local Kubernetes cluster initialized with minikube.

You can refer to the Chapter08/08-end folder in the code repository accompanying the book to check the final result (https://github.com/ThomasVitale/cloud-native-spring-in-action). You can deploy the backing services from the manifests available in the Chapter08/08-end/polar-deployment/kubernetes/platform/development folder with `kubectl apply -f services`.